



Security Assessment Report

Marginfi v2 Program

PR 661

September 04, 2026

Summary

The Sec3 team was engaged to conduct an incremental review of the PR661 of the Marginfi v2 Program.

The artifact of the audit was the source code of the following programs, excluding tests, in <https://github.com/0dotxyz/marginfi-v2>.

The initial audit focused on the following versions and revealed 4 issues or questions.

Task	Type	Commit
PR661: Scope	Solana	bac4029 → bbba606

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview 3

Findings in Detail 4

 [I-01] Missing scope oracle asset_tag compatibility validation 4

 [I-02] Scope oracle and bank mint binding may be inconsistent 6

 [I-03] Scope oracle future timestamps can pass the staleness check 8

 [I-04] A Scope bank with zero oracle_max_age accepts only same-second prices 10

Appendix: Methodology and Scope of Work 12

Result Overview

Issue	Impact	Status
PR661: SCOPE		
[I-01] Missing scope oracle asset_tag compatibility validation	Info	Resolved
[I-02] Scope oracle and bank mint binding may be inconsistent	Info	Acknowledged
[I-03] Scope oracle future timestamps can pass the staleness check	Info	Resolved
[I-04] A Scope bank with zero oracle_max_age accepts only same-second prices	Info	Acknowledged

Findings in Detail

PR661: SCOPE

[I-01] Missing scope oracle asset_tag compatibility validation

Identified in commit [bbba606](#).

`lending_pool_configure_bank_oracle_scope` does not validate whether `OracleSetup::Scope` is compatible with `bank.config.asset_tag`. Its call to `validate_oracle_setup(ctx.remaining_accounts, ...)` only validates the one Scope account supplied to the configuration instruction, so the instruction succeeds for any bank.

At runtime, the oracle-side account count is derived from `asset_tag` rather than from the oracle setup. Health checks call `get_health_components`, which uses `get_remaining_accounts_per_bank(&bank)` to split each bank's account group into the bank account and `oracle_ais`. The same helper drives the split in other operations like `liquidate`:

```
/* programs/marginfi/src/state/marginfi_account.rs */
042 | pub fn get_remaining_accounts_per_bank(bank: &Bank) -> MarginfiResult<usize> {
043 |     match bank.config.oracle_setup {
044 |         OracleSetup::Fixed => Ok(1),
045 |         // Fixed + Kamino: bank + reserve (no oracle)
046 |         OracleSetup::FixedKamino => Ok(2),
047 |         // Fixed + Drift: bank + spot market (no oracle)
048 |         OracleSetup::FixedDrift => Ok(2),
049 |         // Fixed + JupLend: bank + lending state (no oracle)
050 |         OracleSetup::FixedJuplend => Ok(2),
051 |         _ => get_remaining_accounts_per_asset_tag(bank.config.asset_tag),
052 |     }
053 | }

/* programs/marginfi/src/state/marginfi_account.rs */
057 | fn get_remaining_accounts_per_asset_tag(asset_tag: u8) -> MarginfiResult<usize> {
058 |     match asset_tag {
059 |         ASSET_TAG_DEFAULT | ASSET_TAG_SOL => Ok(2),
060 |         ASSET_TAG_KAMINO | ASSET_TAG_DRIFT | ASSET_TAG_SOLEND | ASSET_TAG_JUPLEND => Ok(3),
061 |         ASSET_TAG_STAKED => Ok(5),
062 |         _ => err!(MarginfiError::AssetTagMismatch),
063 |     }
064 | }
```

Each count includes the bank account itself, so `DEFAULT` and `SOL` yield one `oracle_ais` account, the integration tags yield two, and `STAKED` yields four.

However, the Scope branch accepts only `ais.len() == 1`.

```

/* programs/marginfi/src/state/price.rs */
367 | OracleSetup::Scope => {
368 |     check!(ais.len() == 1, MarginfiError::WrongNumberOfOracleAccounts);
369 |     check_primary_oracle_key(bank_config, &ais[0])?;

/* programs/marginfi/src/state/price.rs */
301 | impl OraclePriceFeedAdapter {
320 |     fn load_oracle_context_with_max_age<'info>(
326 |     ) -> MarginfiResult<OracleLoadContext> {
367 |         OracleSetup::Scope => {
368 |             check!(ais.len() == 1, MarginfiError::WrongNumberOfOracleAccounts);
369 |             check_primary_oracle_key(bank_config, &ais[0])?;

```

An administrator can therefore configure a valid Scope feed for an integration or staked bank. A later health operation constructs a two-account or four-account `oracle_ais` slice and returns `WrongNumberOfOracleAccounts`, preventing the affected bank from completing operations that require a live health check.

Every marginfi account with an active balance in the affected bank cannot borrow, withdraw, or complete a liquidation because these operations invoke a live health calculation that fails on the malformed Scope account group, until the administrator restores a compatible oracle setup.

It is recommended to add an oracle-setup and asset-tag compatibility check to both `lending_pool_configure_bank_oracle` and `lending_pool_configure_bank_oracle_scope`. The Scope branch must accept only `ASSET_TAG_DEFAULT` and `ASSET_TAG_SOL`.

Resolution

Resolved by commit [99d8236](#).

PR661: SCOPE

[I-02] Scope oracle and bank mint binding may be inconsistent

Identified in commit [bbba606](#).

`OracleSetup::Scope` identifies a price using `(OraclePrices account, entry_index)`. Neither configuration nor price loading establishes that the entry prices `bank.mint`.

1. The configured index is not checked against the bank mint

`lending_pool_configure_bank_oracle_scope` accepts an arbitrary `entry_index` for an existing bank, writes the pair to the bank configuration, and calls `validate_oracle_setup`, which validates only that the entry is readable:

```
/* programs/marginfi/src/instructions/marginfi_group/config_bank_oracle.rs */
0096 | bank.config.oracle_setup = OracleSetup::Scope;
0097 | bank.config.oracle_keys[0] = oracle;
0098 | bank.config.scope_entry_index = entry_index;
0106 | bank.config
0107 |     .validate_oracle_setup(ctx.remaining_accounts, None, None, None)?;
```

A wrong index chosen here produces no error at any later point.

2. The binding is not re-derived when the price is loaded

`read_entry` verifies the container and the slot's plausibility. But it does not verify the identity:

```
/* marginfi-v2/programs/marginfi/src/state/price.rs */
1382 | impl ScopePriceFeed {
1385 |     fn read_entry(ai: &AccountInfo, entry_index: u16) -> MarginfiResult<Self> {
1402 |         let entry = oracle_prices
1403 |             .prices
1404 |             .get(entry_index as usize)
1405 |             .ok_or(MarginfiError::ScopeInvalidEntry)?;
1407 |         // A never-refreshed entry is all zeroes; reject rather than reporting a price of 0.
1408 |         check!(
1409 |             entry.price.value > 0 && entry.unix_timestamp > 0,
1410 |             MarginfiError::ScopeInvalidEntry
1411 |         );
1412 |         // `exp` indexes the power-of-ten table; anything larger is a malformed entry.
1413 |         check!(
1414 |             (entry.price.exp as usize) < MAX_EXP_10_I80F48,
1415 |             MarginfiError::ScopeInvalidEntry
1416 |         );
```

`OraclePrices` and `OracleMappings` do not embed a mint or token-pair identity for an entry. Scope separately supports an optional `MintsToScopeChains` account that records a mint and a four-entry Scope

chain for a given `OraclePrices` account. This account is a separate admin-created attestation, not data enforced by the price entry itself.

Therefore, the binding may not always be available from Scope data. In such cases, the intended association between a bank mint and a Scope price entry can only be reviewed through an off-chain governance process.

3. The feed admin can repoint the index after configuration

The Scope configuration admin can change the feed's mapping through [update_mapping_and_metadata](#).

Scope zeros the price slot on a repoint, so marginfi may briefly fail with `ScopeInvalidEntry`, as `refresh_price_list` requires no signer so that anybody can repopulate the slot in the next slot. The window in which marginfi can notice is one crank wide.

If the scope configuration admin is outside the trust boundary, the corresponding Scope configuration and mapping data should be monitored after configuration for changes to the selected index and its dependencies.

Resolution

The team acknowledged this finding and clarified that all oracle updates are performed through a multisig, with the Scope feed authority also expected to be controlled by the multisig.

While the team agreed that a mint map could provide an additional safeguard, it is governed by the same operational process and would introduce additional overhead when changes are required, as the account would need to be closed and recreated. The team may consider adding a mint map in a future release.

PR661: SCOPE

[I-03] Scope oracle future timestamps can pass the staleness check

Identified in commit [bbba606](#).

`ScopePriceFeed::load_checked` accepts an entry when its age (`i64`) is no greater than the bank's `oracle_max_age`. It does not require the Scope entry timestamp to be at or before the current timestamp.

```
/* programs/marginfi/src/state/price.rs */
1428 | pub fn load_checked(
1429 |     ai: &AccountInfo,
1430 |     current_timestamp: i64,
1431 |     max_age: u64,
1432 |     entry_index: u16,
1433 | ) -> MarginfiResult<Self> {
1434 |     let feed = Self::read_entry(ai, entry_index)?;
1436 |     let age = current_timestamp.saturating_sub(feed.last_updated_timestamp as i64);
1437 |     check!(age <= max_age as i64, MarginfiError::ScopeStalePrice);
1439 |     Ok(feed)
1440 | }
```

When `feed.last_updated_timestamp` is in the future, `age` is negative and satisfies the upper-bound check. `ScopePriceFeed::read_entry` does not reject a future timestamp before this check.

```
/* programs/marginfi/src/state/price.rs */
1382 | impl ScopePriceFeed {
1385 |     fn read_entry(ai: &AccountInfo, entry_index: u16) -> MarginfiResult<Self> {
1402 |         let entry = oracle_prices
1403 |             .prices
1404 |             .get(entry_index as usize)
1405 |             .ok_or(MarginfiError::ScopeInvalidEntry)?;
1407 |         // A never-refreshed entry is all zeroes; reject rather than reporting a price of 0.
1408 |         check!(
1409 |             entry.price.value > 0 && entry.unix_timestamp > 0,
1410 |             MarginfiError::ScopeInvalidEntry
1411 |         );
1412 |     }
1413 | }
```

Scope provides `clamp_timestamp_to_now` and uses it in certain oracle adapters to cap a source timestamp at the current timestamp.

```
/* https://github.com/Kamino-Finance/scope/blob/master/programs/scope/src/utils/math.rs */
292 | pub fn clamp_timestamp_to_now(
293 |     timestamp: i64,
294 |     clock: &solana_program::clock::Clock,
295 | ) -> ScopeResult<u64> {
296 |     let clamped_timestamp = timestamp.min(clock.unix_timestamp);
297 |     u64::try_from(clamped_timestamp).map_err(|_| ScopeError::BadTimestamp)
298 | }
```

However, this is a **per-adapter behavior** rather than a guarantee of the `DatedPrice` layout, and computed

types derive their timestamp from source entries. If a Scope price-update path does not call `clamp_time stamp_to_now`, it can write a future timestamp into `OraclePrices`.

Consider rejecting a Scope timestamp that is later than the current timestamp before applying the maximum-age check.

Resolution

Resolved by commit [99d8236](#).

PR661: SCOPE

[I-04] A Scope bank with zero oracle_max_age accepts only same-second prices

Identified in commit [bbba606](#).

New bank configuration is expected to set a non-zero oracle freshness window, as the bank config validator enforces `oracle_max_age >= ORACLE_MIN_AGE`.

```
/* programs/marginfi/src/state/bank_config.rs */
050 | fn validate(&self) -> MarginfiResult {
080 |     check!(
081 |         self.oracle_max_age >= ORACLE_MIN_AGE,
082 |         MarginfiError::InvalidOracleSetup
083 |     );
```

Commit [f5cfba8](#) introduced the `ORACLE_MIN_AGE` validation after banks with `oracle_max_age = 0` had already been created on mainnet. Currently, 84 mainnet bank accounts still store a zero `oracle_max_age`.

```
/* programs/marginfi/src/state/bank_config.rs */
172 | fn get_oracle_max_age(&self) -> u64 {
173 |     match (self.oracle_max_age, self.oracle_setup) {
174 |         (0, OracleSetup::PythPushOracle) => MAX_PYTH_ORACLE_AGE,
175 |         (n, _) => n as u64,
176 |     }
177 | }
```

For legacy `PythPushOracle` banks with zero `oracle_max_age`, the current read path maps `0` to `MAX_PYTH_ORACLE_AGE` when the oracle setup is `PythPushOracle`.

Now, the Scope oracle setter can move an existing bank with `oracle_max_age = 0` to Scope, while preserving its stored `oracle_max_age`. During the configuration, it validates only the supplied Scope account and entry.

```
/* programs/marginfi/src/instructions/marginfi_group/config_bank_oracle.rs */
076 | pub fn lending_pool_configure_bank_oracle_scope(
077 |     ctx: Context<LendingPoolConfigureBankOracle>,
078 |     oracle: Pubkey,
079 |     entry_index: u16,
080 | ) -> MarginfiResult {
106 |     bank.config
107 |         .validate_oracle_setup(ctx.remaining_accounts, None, None, None)?;
```

When Scope is configured for a bank that retains `oracle_max_age = 0`, the `PythPushOracle` compatibility fallback no longer applies. The resolved Scope maximum age is therefore zero. A Scope entry published

in an earlier timestamp fails with `ScopeStalePrice`. Only an entry from the current second passes the normal freshness check.

```
/* programs/marginfi/src/state/price.rs */
1428 | pub fn load_checked(
1429 |     ai: &AccountInfo,
1430 |     current_timestamp: i64,
1431 |     max_age: u64,
1432 |     entry_index: u16,
1433 | ) -> MarginfiResult<Self> {
1434 |     let feed = Self::read_entry(ai, entry_index)?;
1436 |     let age = current_timestamp.saturating_sub(feed.last_updated_timestamp as i64);
1437 |     check!(age <= max_age as i64, MarginfiError::ScopeStalePrice);
```

This can make the affected bank's Scope price unusable until the administrator sets a valid `oracle_max_age`.

It is recommended to add a Scope compatibility branch to `get_oracle_max_age` or reject `oracle_max_age = 0` in `lending_pool_configure_bank_oracle_scope`.

Resolution

The team acknowledged this finding and clarified that the 60-second fallback for Pyth banks was introduced solely for migration purposes and is no longer required, as all production banks have been migrated. The 84 identified banks are test or otherwise unused banks outside the main group.

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and MRGN, Inc. (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

